1 Introduction

We containerised the infrastructure outlined in the laboratory assignment: the shared services (*basic*), a company application (*app*), an observability platform (*stats*), and Moodle (*atenea*). For the core application, we developed a reviews service in Flask: users can post comments with a 1-to-5 rating, view score distributions and recent reviews, trigger email notifications through an internal SMTP relay upon each submission, and ingest external ratings via an automated background feeder.

The implementation is designed for production-grade resilience: every user-facing component maintains at least two replicas distributed across distinct nodes, the application database and caching layers are clustered with automatic failover, backups are encrypted and stored on a dedicated node, and all services export health and telemetry metrics. Deployment is fully automated, requiring only a single `docker compose up` for local execution or `docker stack deploy` for production Swarm orchestration, using pre-built images published to Docker Hub.

The repository is organised into `compose.yml` (local), `production.yml` (Swarm), service directories containing their respective `Dockerfile` definitions and runtime assets, helper automation in `scripts/` (node labelling, image publishing, Swarm simulation, and disaster recovery), and this report. The local environment operates without TLS and uses named volumes, yet preserves the exact clustering topology present in production to enable verification on a single development machine.

2 Architecture

2.1 Nodes

The swarm comprises seven nodes; service placement is controlled via the Docker Swarm node label `role` (along with `slot` on the application nodes to enforce separation between primaries and replicas).

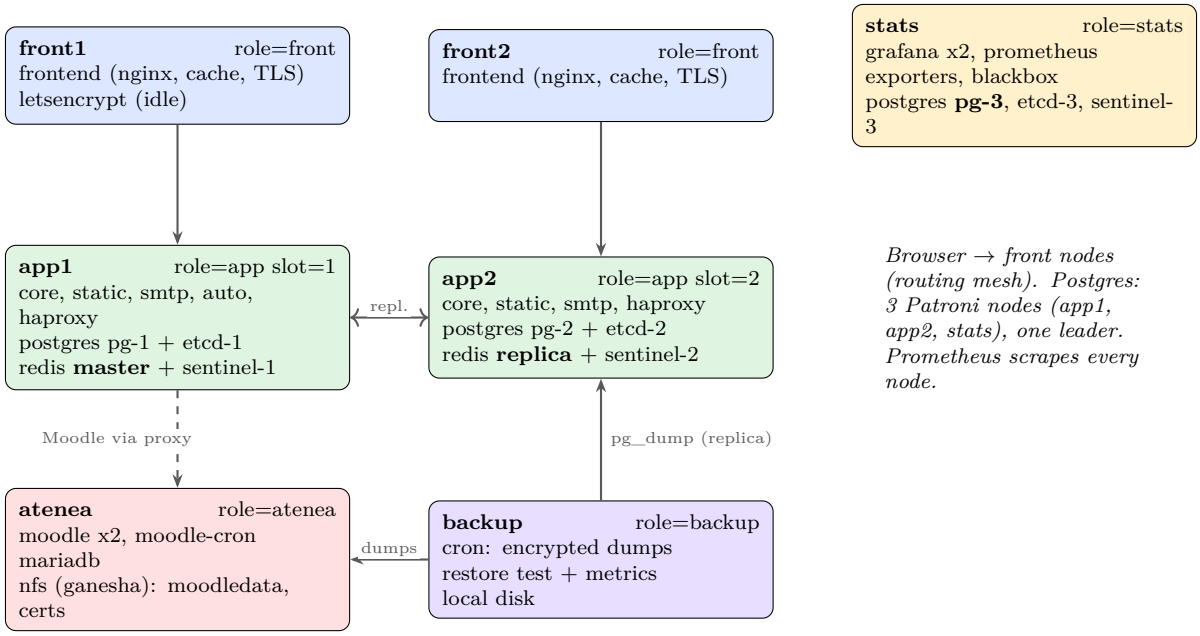


Figure 1: Nodes and services (simplified). The routing mesh accepts the traffic on both front nodes; `backup` pulls from a replica, from MariaDB and from the NFS volume.

2.2 Services

Table 1 maps the components from the assignment specification to our implemented services, the nodes where they run, and their replica counts. Each stack operates within an encrypted overlay

network, and each service only joins the networks it strictly requires: for instance, the databases do not participate in the `proxy` network, ensuring they remain isolated and inaccessible from the frontend.

Stack	Service	Nodes	Repl.	Description
basic	`frontend`	front	2	nginx: only public service (80/443), cache, TLS, proxy by subdomain
	`letsencrypt`	front	1	certbot; idle with a local domain (self-signed certificate)
	`nfs`	atenea	1	NFSv4 server (NFS-Ganesha) inside the stack, port 2049 in host mode
app	`core`	app	2	Flask + gunicorn, reviews
	`static`	app	2	Apache httpd, `/static/`
	`pg-1..3`	app1, app2, stats	3	PostgreSQL 16 + Patroni (leader + 2 replicas)
	`etcd-1..3`	app1, app2, stats	3	consensus store of Patroni
	`db`	app	2	HAProxy: `db:5432` leader, `db:5433` replicas
	`redis-1`, `redis-2`	app1, app2	1+1	cache: master and replica
	`sentinel-1..3`	app1, app2, stats	3	failover of the cache (quorum 2)
	`smtp`	app	2	Postfix relaying the mails (to Mailpit by default)
	`mailpit`	app	1	mail catcher with a web page (`mail.localhost:8080`)
	`auto`	app	1	feeder of external ratings
	`backup`	backup	1	encrypted dumps + restore test
atenea	`moodle`	atenea	2	Moodle 4.5.14 LTS
	`moodle-cron`	atenea	1	same image, `cron.php` every minute
	`moodle-db`	atenea	1	MariaDB 11.4
stats	`grafana`	stats	2	dashboards as code + Infinity plugin (the "?")
	`prometheus`	stats, backup	2	metrics and alert rules, one TSDB per replica
	exporters	stats / global	—	postgres, redis, mysqld, nginx, blackbox, node, cAdvisor

Table 1: Services of the infrastructure.

Public entrypoints in production are exposed via `app.<domain>`, `lms.<domain>`, and `grafana.<domain>`; with the default `DOMAIN=localhost`, they resolve to `app.localhost`, `lms.localhost`, and `grafana.localhost` on port 443. Locally, the same names are served on port 8080 over HTTP (modern browsers resolve `*.localhost` to 127.0.0.1 automatically).

3 Images and design decisions

Frontend and caching. We use `nginx:alpine` as the single public entrypoint for all services. Nginx terminates TLS, routes subdomains to their internal services, and caches dynamic pages for 10 seconds and static files for one hour. When a user submits a review, a temporary cookie bypasses the cache so their new comment appears immediately. The page footer displays which frontend and application nodes handled the request, demonstrating load distribution across replicas.

TLS and certificates. Production exposes HTTPS on port 443 using a self-signed certificate generated at container startup, allowing Nginx to boot without external dependencies. The `letsencrypt` service is fully defined in the stack with automated renewal logic, but remains idle when running with local domains (`localhost`) where public certificate authorities cannot validate ownership.

Core application. The reviews application is built with Python 3.12 and runs as a non-root user with two replicas. To maintain high performance, page data is cached in Redis with version-based invalidation upon new submissions. The application degrades gracefully: if Redis is unreachable, requests fall back directly to the database; if the database is temporarily recovering, the service returns a clean HTTP 503 instead of crashing. Database operations are decoupled via HAProxy: writes target the leader (port 5432) while reads are distributed across replicas (port 5433).

PostgreSQL cluster (Patroni). Database high availability is managed by Patroni across three PostgreSQL 16 nodes (`pg-1`, `pg-2`, `pg-3`) distributed over `app1`, `app2`, and `stats`. A three-node etcd cluster provides distributed consensus for leader election and health tracking. If the active leader fails, Patroni automatically promotes a replica within seconds. HAProxy (`db`) sits in front of the cluster, continuously querying Patroni's health API to expose consistent endpoints (`db:5432` for writes, `db:5433` for reads) so application services require no configuration changes during a failover.

Redis cache and Sentinel. Redis caching uses a primary-replica architecture paired with three Redis Sentinels across separate nodes (quorum of 2). The application dynamically discovers the active master through Sentinel, enabling automatic failover if the primary node goes offline.

Moodle LMS. Moodle 4.5 LTS is deployed using PHP 8.3 Apache with two replicas. On initial launch, an automated script runs the command-line installer, utilizing a filesystem lock in the shared volume to prevent race conditions between containers. User sessions are persisted in MariaDB, allowing any replica to serve incoming requests interchangeably. A dedicated container (`moodle-cron`) runs scheduled tasks in the background.

Shared storage (NFS). Because Docker Swarm restricts privileged containers on worker nodes, we deploy **NFS-Ganesha** in user space on the `atenea` node. It exports shared volumes for Moodle data and certificates using only standard Linux capabilities, enabling seamless multi-node file sharing without requiring kernel-level NFS modules on the host.

Backups and observability. See Sections **??** and **??**.

Scope of production. In the context of this project, "production" represents the complete multi-node Swarm architecture running on local Docker-in-Docker nodes. Because it is evaluated locally rather than on a public cloud provider, services use `localhost` subdomains and self-signed certificates.

4 Scalability and high availability

4.1 Replicas

The project guidelines require all services to be scalable, excluding periodic jobs and non-distributed DBMS. Stateless components (`frontend`, `core`, `static`, `db`, `smtp`, `grafana`, `moodle`, and `prometheus`) are deployed with two replicas and can be dynamically scaled using `docker service scale`. Prometheus automatically discovers active replicas via Docker Swarm DNS (`tasks.<service>`).

Services intentionally kept at a single replica include periodic background jobs (`letsencrypt`, `backup`, `moodle-cron`, and `auto`, where duplicate runs would be redundant), single-host storage (`nfs`), developmental tooling (`mailpit`), and `moodle-db`. The primary stateful layers (PostgreSQL and Redis) are not scaled naively, but are deployed as high-availability clusters with automated leader election and replication.

4.2 Failures

Table 2 summarizes the system resilience matrix, categorizing the direct impact, mitigation mechanism, and recovery process across service-level and node-level outages.

Failure Target	Direct Impact	Mitigation Mechanism	Recovery
Service / Container Failures			
Frontend replica	None	Swarm routing mesh directs traffic to survivor	Automatic
Core / static replica	None (transparent)	Nginx serves stale cache; Swarm reschedules	Automatic
Redis master	Brief cache miss spike	Sentinels promote replica; app degrades gracefully	Automatic (<5 s)
Postgres leader	Brief write pause	Patroni elects replica; HAProxy reroutes traffic	Automatic (<3 s)
Node Outages			
`front1` or `front2`	None	Routing mesh routes via surviving front node	Automatic
`app1` or `app2`	Reduced capacity	App2 serves; Patroni/Redis failover; etcd 2/3 quorum	Automatic
`stats`	Metrics interrupted	Core app unaffected; etcd/Patroni maintain 2/3 quorum	On node restart
`atenea`	Moodle & NFS down	Core app unaffected; TLS certs cached in Nginx	On node restart
`backup`	Backup jobs pause	User-facing services completely unaffected	On node restart

Table 2: System resilience and failure recovery matrix.

4.3 Updates

Updates are applied one container at a time (`parallelism: 1`, `order: stop-first`) with automated rollback upon error (`failure_action: rollback`). Each custom image includes a defined `HEALTHCHECK`, guaranteeing that an updated replica is verified healthy before the deployment progresses to the next node. Furthermore, stateless services enforce `max_replicas_per_node: 1`, ensuring replicas reside on distinct physical hosts so that a single host failure never compromises service availability.